

OPC UA Communication Engine

System Architecture Document

.NET 8.0 WPF + ASP.NET Core REST API + SignalR

Version: 1.0.0

Date: 2026-04-12

Framework: .NET 8.0 (Windows)

UI: WPF (Windows Presentation Foundation)

API: ASP.NET Core + SignalR

Protocols: OPC UA, Modbus TCP, Siemens S7, Mitsubishi MC

1. Overview

OPC UA Communication Engine is a multi-PLC industrial communication platform supporting 4 protocols: OPC UA, Modbus TCP, Siemens S7, and Mitsubishi MC. It provides a WPF desktop interface for monitoring and an embedded ASP.NET Core server exposing REST API and SignalR Hub for mobile/web clients.

Key features: multi-protocol PLC connections, real-time tag monitoring via polling/subscriptions, JWT authentication with role-based access, operator lock mechanism for write safety, automatic reconnection on connection loss, and SignalR-based real-time data broadcasting.

2. Project Structure

```
OPCUACommDriver/
|-- Interfaces/           # Service contracts
|-- Models/              # Data models
|-- Enums/               # Enumerations
|-- Services/
|   |-- OpcUa/            # PlcManager, PlcConnection (OPC UA)
|   |-- Protocols/        # BaseProtocolConnection, Modbus/S7/MC
|   |-- Auth/             # AuthService, UserService, OperatorLockService
|   |-- ConfigurationService.cs
|   |-- DataCacheService.cs
|-- ViewModels/          # MVVM ViewModels
|-- Views/               # WPF Windows & Dialogs
|-- Helpers/             # Converters, RelayCommand
|-- Api/
|   |-- Controllers/      # REST (Plcs, Tags, Auth, Users, Lock)
|   |-- Hubs/             # SignalR PlcHub
|   |-- Models/           # API DTOs
|   |-- ApiHostService.cs
|-- Configurations/      # JSON config files
```

3. Architecture Layers

3.1 Layer Overview

Layer	Components	Description
UI	WPF Views + MVVM	MainWindow, LoginWindow, Dialogs
ViewModel	MainViewModel	MVVM binding, commands, state
Business	PlcManager, DataCache	Connection mgmt, caching, config
Protocol	PlcConnection, Base*	OPC UA, Modbus, S7, MC
API	Controllers + SignalR	REST endpoints + WebSocket hub
Auth	JWT + bcrypt + Lock	Authentication, authorization
Config	JSON files	plc_config, appsettings, users

3.2 Protocol Connection Hierarchy

```
IPlcConnection (interface)
|-- PlcConnection          [OPC UA direct implementation]
|   Session, KeepAlive, Subscriptions
|
|-- BaseProtocolConnection  [Abstract base for TCP protocols]
|   Background Polling, Connection Detection,
|   Auto-Reconnect, SemaphoreSlim Lock
|
|-- ModbusTcpConnection     [Modbus TCP/IP - port 502]
|   FC01-FC06, FC16, Bit-level HR access
|
```

```
|-- SiemensS7Connection    [S7comm - port 102]
|    DB, MW, IW, QW memory areas
|
|-- MitsubishiMcConnection [MC Protocol - port 5000]
|    D, W, M, X, Y device types
```

4. Data Flow

4.1 Tag Value Read Flow (PLC -> Client)

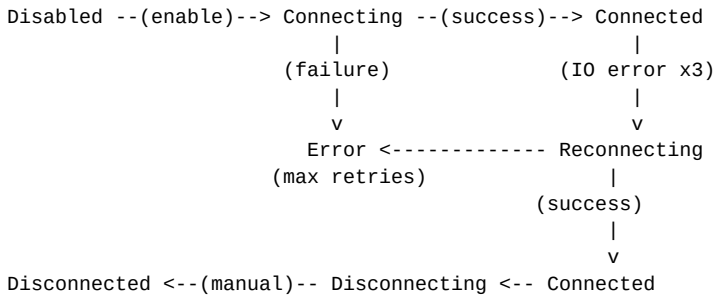
```
PLC Device
| (Modbus TCP / OPC UA / S7 / MC)
v
Protocol Connection (poll or subscribe)
| ReadTagInternalAsync() -> TagValue
v
tag.UpdateValue() + OnTagValueChanged event
|
v
PlcManager.TagValueChanged event
|
+---> DataCacheService.SetValue()
|
+---> ApiHostService subscriber
|
v
PlcHubService.BroadcastTagValueAsync()
|
v
SignalR -> "TagValueChanged" -> All clients
```

4.2 Tag Value Write Flow (Client -> PLC)

```
Client (Android/Web)
| POST /api/tags/{plcId}/{nodeId}/write
v
TagsController -> JWT Auth check
v
OperatorLock check (must hold lock)
v
PlcManager.WriteTagAsync()
v
IPlcConnection.WriteTagAsync()
| (SemaphoreSlim lock for TCP protocols)
v
WriteTagInternalAsync() -> PLC Device
```

5. Connection State Management

5.1 State Machine



5.2 Connection State Broadcast Chain

```
BaseProtocolConnection.ConnectionState setter
| fires OnConnectionStateChanged event
v
PlcManager.OnConnectionStateChanged()
| re-fires ConnectionStateChanged event
v
ApiHostService subscriber
| calls PlcHubService.BroadcastConnectionStateAsync()
v
SignalR -> "ConnectionStateChanged" message
| {PlcId, PlcName, State, IsConnected, LastStateChange}
v
ALL connected clients (Android, Web, etc.)
```

5.3 Modbus TCP Connection Loss Detection

The polling loop in `BaseProtocolConnection` tracks consecutive IO/Socket errors. When ALL tags fail with `IOException` or `SocketException` for 3 consecutive polling cycles, `HandleConnectionLostAsync()` is triggered:

- Marks all tags quality = Bad
- Calls `DisconnectInternalAsync()` to cleanup dead TCP socket
- Fires `ErrorOccurred` event (broadcast via SignalR)
- Calls `StartAutoReconnectAsync()` with configurable interval/retries
- State transitions: `Connected -> Reconnecting -> Connected` or `Error`

6. REST API Endpoints

6.1 PLC Management

Method	Endpoint	Description
GET	/api/plcs	List all PLCs with connection state
GET	/api/plcs/{id}	Get specific PLC details
GET	/api/plcs/status	Connection status of all PLCs
POST	/api/plcs/{id}/connect	Connect to a PLC
POST	/api/plcs/{id}/disconnect	Disconnect from a PLC
POST	/api/plcs/connect-all	Connect to all PLCs
POST	/api/plcs/disconnect-all	Disconnect all PLCs
GET	/api/plcs/{id}/browse	Browse OPC UA server nodes

6.2 Tag Operations

Method	Endpoint	Description
GET	/api/tags	List all tags
GET	/api/tags/plc/{plcId}	Tags for specific PLC
GET	/api/tags/subscribed	Tags from connected PLCs only
GET	/api/tags/{plcId}/{nodeId}/read	Read single tag value
POST	/api/tags/{plcId}/{nodeId}/write	Write single tag value
POST	/api/tags/{plcId}/write-multiple	Write multiple tags

6.3 Authentication & Lock

Method	Endpoint	Description
POST	/api/auth/login	Login (returns JWT access + refresh tokens)
POST	/api/auth/logout	Logout and revoke session
POST	/api/auth/refresh	Refresh access token
GET	/api/users	List users (Admin only)
POST	/api/lock/acquire	Acquire operator lock for writing
POST	/api/lock/release	Release operator lock
GET	/api/lock/status	Check current lock status

7. SignalR Real-time Hub

Hub URL: /hubs/plc (WebSocket with JWT authentication)

7.1 Server -> Client Messages

Message	Payload
ConnectionStateChanged	{PlcId, PlcName, State, IsConnected, LastStateChange}
TagValueChanged	{PlcId, TagId, NodeId, Value, Quality, Timestamp}
PlcStatus	List<{PlcId, PlcName, State, IsConnected}>
LockAcquired	{LockId, UserId, Username, AcquiredAt, ExpiresAt}
LockReleased	{LockId, ReleasedBy}
LockExtended	{LockId, NewExpiresAt}

7.2 Client -> Server Methods

Method	Description
SubscribeToPlc(plcId)	Subscribe to updates for a specific PLC
UnsubscribeFromPlc(plcId)	Unsubscribe from a specific PLC
SubscribeToAll()	Subscribe to all PLC updates
GetStatus()	Request current connection status of all PLCs
GetAllTagValues()	Request all current tag values
WriteTag(plcId, nodeId, value)	Write a tag value via SignalR

8. Authentication & Security

8.1 Role-Based Access Control

Role	Read	Write	Lock	User Mgmt	Notes
Viewer	Yes	No	No	No	Read-only access
Operator	Yes	Yes*	Yes	No	*Requires lock
Admin	Yes	Yes	Override	Yes	Full access

8.2 Authentication Flow

```
Client                                     Server
|                                         |
|--- POST /api/auth/login ----->|
|   {username, password,             |
|   deviceId}                       |
|                                     |
|<-- {accessToken,                  |
|     refreshToken,                 |
|     expiresAt} ----->|
|                                     |
|--- GET /api/tags ----->|
|   Authorization: Bearer token      |
|                                     |
|--- POST /api/lock/acquire ----->|
|   (Operator/Admin only)            |
|                                     |
|--- POST /api/tags/.../write -->|
|   (Must hold operator lock)        |
|                                     |
```

8.3 Operator Lock Mechanism

Only one operator can write at a time. The lock has configurable timeout and auto-expiration. Admin can override locks. Lock state is persisted to lock_state.json and broadcast via SignalR on acquire/release/extend.

9. Modbus TCP Tag Mapping

9.1 Address Format

Format	Description	Example
HR{n}	Holding Register n (FC03/FC06)	HR0, HR1, HR100
HR{n}.{bit}	Bit in Holding Register	HR0.0, HR0.14
IR{n}	Input Register n (FC04)	IR0, IR50
C{n}	Coil n (FC01/FC05)	C0, C8192
DI{n}	Discrete Input n (FC02)	DI0, DI50

9.2 S7-1500 Mapping (192.168.1.10:502)

Booleans packed in HR0 bits, integers in HR1-HR28:

Tag	NodeId	DataType	PLC Register	Modbus Addr
bool1	HR0.0	Boolean	MB_HOLD_REG[0].0	40001 bit 0
bool2	HR0.1	Boolean	MB_HOLD_REG[0].1	40001 bit 1
...	...	...	...	...
bool15	HR0.14	Boolean	MB_HOLD_REG[0].14	40001 bit 14
int1	HR1	Int16	MB_HOLD_REG[1]	40002
int2	HR2	Int16	MB_HOLD_REG[2]	40003
...	...	...	...	...
int28	HR28	Int16	MB_HOLD_REG[28]	40029

9.3 FX5U Mapping (192.168.1.11:502)

Booleans via M coils (direct bit), integers via D registers:

Tag	NodeId	DataType	PLC Device	Modbus Addr
bool1	C8192	Boolean	M0	Coil 8192
bool2	C8193	Boolean	M1	Coil 8193
...	...	...	...	...
bool15	C8206	Boolean	M14	Coil 8206
int1	HR0	Int16	D0	40001
int2	HR1	Int16	D1	40002
...	...	...	...	...
int28	HR27	Int16	D27	40028

10. Dependencies

Package	Version	Purpose
OPCFoundation.NetStandard.Opc.Ua	1.5.374	OPC UA protocol stack
OPCFoundation...Opc.Ua.Client	1.5.374	OPC UA client library
CommunityToolkit.Mvvm	8.2.2	MVVM toolkit for WPF
Newtonsoft.Json	13.0.3	JSON serialization
Serilog + Sinks	3.1.1	Structured logging (File, Console)
Microsoft.Extensions.DI	8.0.0	Dependency Injection
Microsoft.AspNetCore.App	8.0	ASP.NET Core framework
Swashbuckle.AspNetCore	6.5.0	Swagger / OpenAPI docs
BCrypt.Net-Next	4.0.3	Password hashing
MS...Authentication.JwtBearer	8.0.0	JWT authentication

11. Configuration Files

File	Purpose
plc_config.json	PLC devices, tags, subscription groups
appsettings.json	API port, bind address, logging settings
auth_settings.json	JWT secret, token expiry, lock timeout
users.json	User accounts (bcrypt hashed passwords)
lock_state.json	Current operator lock state (auto-generated)
modbus_plc_config_sample.json	Sample config for Modbus TCP PLCs